



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4306 · Design & Analysis of Algorithms · Fall 2026

Counting Loop Nests

Sequences add, nests multiply, and dependent nests need the series

Session 5 of 24 · Friday 2 October 2026 · 8:30 – 9:50 AM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 5 OF 24

Counting Loop Nests

Friday, 2 October 2026 · 8:30 – 9:50 AM

LEARNING OBJECTIVES

- ✓ Apply the three counting rules: sequence, nesting, dependence.
- ✓ Explain why a dependent nest that does “half the work” is still quadratic.
- ✓ Recognise that j^2 and $j + 1$ produce different classes.
- ✓ Read a library call as the loop it really is.
- ✓ Give Θ for an unfamiliar loop nest by counting rather than by pattern-matching.

FROM SESSIONS 3 AND 4

We can now name growth and prove the naming. What we cannot yet do reliably is *extract* the growth from a piece of code that nobody has labelled for us. Today we stop reading complexity off the code by intuition and start counting.

1 · Three Rules

12 min

Sequence → add

Work done one after another adds up. A $\Theta(n)$ pass followed by a $\Theta(n^2)$ pass costs $\Theta(n + n^2) = \Theta(n^2)$.

Nesting → multiply

A block costing b , executed a times, costs $a \cdot b$.

Dependence → sum the series

When the inner bound mentions the outer variable, you may **not** multiply. You must add up what actually runs.

These are not tricks. They are arithmetic, and the third one is where careful students separate themselves from hopeful ones.

2 • The Independent Nest 10 min

PY TWO INDEPENDENT LOOPS

```
def count_A(n):
    ops = 0
    for i in range(n):           # outer: n iterations
        for j in range(n):       # inner: n iterations, every single time
            ops += 1             # body: constant
    return ops                  # returns exactly n * n
```

$\Theta(n^2)$

Rule 2, and nothing more: $n \times n \times \text{constant}$. The inner bound does not mention `i`, so the inner loop is the same length every time round.

Do the same with three nested loops and you get $\Theta(n^3)$. With a nest whose inner bound is a *constant*, you get something different:

PY A NEST THAT IS NOT QUADRATIC

```
def count_B(n):
    ops = 0
    for i in range(n):           # n iterations
        for j in range(10):      # 10 iterations - does not depend on n
            ops += 1
    return ops                  # returns 10 * n
```

$\Theta(n)$

10n operations, and Move 1 discards the 10. **Nesting does not mean quadratic**; it means multiply, and one of the factors here is a constant.

3 · The Dependent Nest

20 min

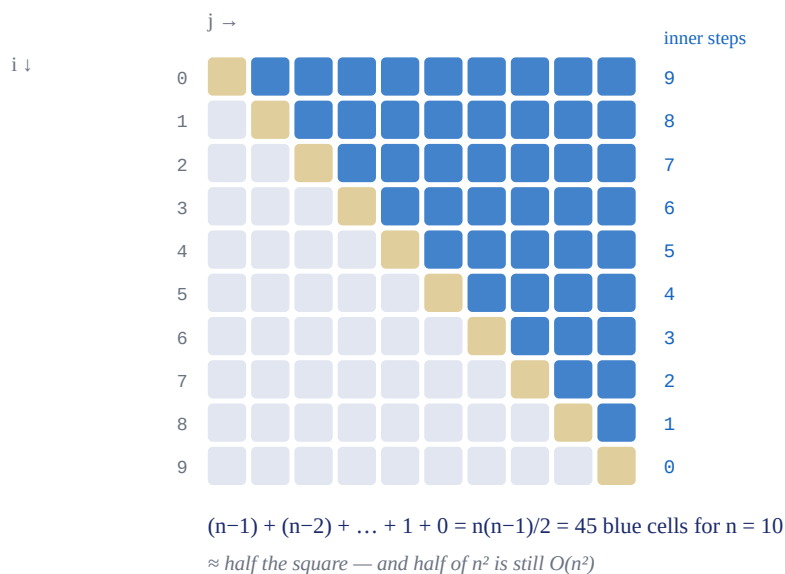
PY THE INNER BOUND DEPENDS ON THE OUTER INDEX

```
def count_C(n):
    ops = 0
    for i in range(n):
        for j in range(i + 1, n):  # starts at i+1, not at 0
            ops += 1
    return ops
```

Now the inner loop is a different length each time. When $i = 0$ it runs $n-1$ times; when $i = 1$, $n-2$ times; and so on down to 0. Rule 3 says add them:

$$(n-1) + (n-2) + \dots + 2 + 1 + 0 = n(n-1)/2$$

Many students conclude: half the work, therefore one class better — $\Theta(n)$. It is not. Half of n^2 is a constant multiple of n^2 , and Move 1 threw constants away.



The triangle. Blue cells are pairs (i, j) the dependent nest actually visits; gold is the diagonal it skips; grey is what it never touches. The blue region is half the square — and half a square still grows like a square. The saving is worth having in practice and worth nothing asymptotically.

TWO FACTS THAT MUST COEXIST IN YOUR HEAD

1. `count_C` does half the work of `count_A`. That is real: it will finish in half the time.
 2. Both are $\Theta(n^2)$. That is also real: doubling n makes *both* take four times as long.
- Big-O never claimed the two functions were equally fast. It claimed they react the same way to growth.

3.1 A useful sum to know by heart

Rule 3 keeps producing the same two series. Memorise them; you will use them all term.

Series	Closed form	Θ	Where it comes from
$1 + 2 + \dots + n$	$n(n+1)/2$	$\Theta(n^2)$	a dependent nest, insertion sort
$1 + 2 + 4 + \dots + n$	$2n - 1$	$\Theta(n)$	doubling, binary tree levels
$1 + \frac{1}{2} + \frac{1}{4} + \dots$	< 2	$\Theta(1)$	halving work at each level

Notice the second and third. A series that *doubles* is dominated by its last term; a series that *halves* is dominated by its first. Only the arithmetic series in the first row genuinely accumulates across all its terms. We will use exactly this observation on recursion trees in Session 10.

4 • When the Counter Is Multiplied 15 min

PY AN INNER LOOP THAT DOUBLES

```
def count_D(n):
    ops = 0
    for i in range(n):           # n iterations
        j = 1
        while j < n:             # j takes values 1, 2, 4, 8, 16, ...
            ops += 1
            j *= 2                # doubling, not incrementing
    return ops
```

$\Theta(n \log n)$

The inner loop runs about $\log_2 n$ times, because doubling from 1 reaches n after $\log_2 n$ steps. Outer n , inner $\log n$, Rule 2 multiplies them.

THE OPERATOR DECIDES THE CLASS

- `j += 1` → the loop runs n times → linear.
- `j *= 2` → the loop runs $\log_2 n$ times → logarithmic.
- `j /= 2` starting from n → also $\log_2 n$ times.
- `j *= 3` → $\log_3 n$ times, which is $\Theta(\log n)$ as well, because changing the base of a logarithm only

multiplies by a constant — and Move 1 discards constants. **All logarithms are the same logarithm as far as Big-O is concerned.**

One more, worth tracing carefully because it looks quadratic and is not:

PY A SHRINKING INNER LOOP

```
def count_E(n):
    ops = 0
    i = n
    while i > 0:
        for j in range(i):
            ops += 1
        i //= 2
    return ops
```

i: n, n/2, n/4, ... -> log n rounds
inner loop runs i times, and i is shrinking

The inner loop runs n times, then $n/2$, then $n/4$, and so on. By the halving series above, the total is $n + n/2 + n/4 + \dots < 2n$.

$O(n)$

A nest of two loops, both depending on n , that is nevertheless **linear**. This is why counting beats pattern matching: “two loops, therefore n^2 ” is wrong here by a whole class.

5 • Loops Hiding Inside Library Calls 15 min

Python lets you write a scan as a single short expression. The scan is still there.

PY THE SAME FUNCTION, TWO COMPLEXITY CLASSES

```
def has_duplicate_slow(A):
    for i in range(len(A)):
        if A[i] in A[i + 1:]:
            return True
    return False

def has_duplicate_fast(A):
    seen = set()
    for v in A:
        if v in seen:
            return True
        seen.add(v)
    return False
```

one line: a full scan AND a copy of the tail
set membership: $O(1)$ on average

Expression	Cost	Why
<code>x in list</code>	$\Theta(n)$	scans until found
<code>x in set / x in dict</code>	$\Theta(1)$ average	hashing, Session 6
<code>A[i:j]</code>	$\Theta(j - i)$	builds a new list; slicing copies
<code>A.append(x)</code>	$\Theta(1)$ amortised	writes at the end
<code>A.insert(0, x) / A.pop(0)</code>	$\Theta(n)$	every element shifts
<code>sorted(A) / A.sort()</code>	$\Theta(n \log n)$	Timsort, Session 11
<code>min(A), max(A), sum(A)</code>	$\Theta(n)$	one pass each
<code>len(A)</code>	$\Theta(1)$	the length is stored, not counted

The most common accidental quadratic in real code

`result = []` then `result.insert(0, x)` inside a loop over n items. It looks linear. It is $\Theta(n^2)$, because every insert shifts everything already there. Use `append` and reverse at the end, or use a `deque`.

THE HABIT TO BUILD

When counting, read every library call and ask: **does this hide a loop over my data?** The visual difference between `has_duplicate_slow` and `has_duplicate_fast` is three lines. The difference in behaviour at $n = 100,000$ is the difference between a coffee break and an instant.

SESSION 5 TAKEAWAY

Sequences add, nests multiply, dependent nests need the series. Nesting alone does not mean quadratic; a constant inner bound gives linear, a doubling counter gives logarithmic, and a shrinking inner loop can give linear from two nested loops. Read library calls as the loops they are. **Count. Do not pattern-match.**

IN-CLASS EXERCISES (18 min — work in pairs)

1. Give Θ for each, and say which rule you used: (a) `for i in range(n): for j in range(5): ...` (b) `for i in range(n): for j in range(i): ...` (c) `for i in`

```
range(n): for j in range(n): for k in range(10): ... (d) i = 1 then while  
i < n: i *= 3
```

2. Count exactly (not just asymptotically) how many times the body runs in `for i in range(n):
for j in range(i, n): ...`
3. Trace `count_E(16)` and report the exact value of `ops`. Does it match the prediction “a little under $2n$ ”?
4. This function looks linear. What is its true Θ , and why? `out = [] ; for v in A:
out.insert(0, v)`
5. Write a function that is $\Theta(n \log n)$ using only loops — no sorting, no recursion, no library calls.

Homework · due Tuesday 6 October

1. Implement all five counting functions from this lecture. For each, tabulate the returned `ops` at $n = 100, 200, 400, 800$ and confirm the reaction predicted by your Θ (unchanged, $\times 2$, $\times 4$, or slightly more than $\times 2$).
2. Time `has_duplicate_slow` and `has_duplicate_fast` on random lists with no duplicates at $n = 1000, 2000, 4000, 8000$. Submit the table and identify each one's class from the measurements alone.
3. Replace the slice `A[i+1:]` in `has_duplicate_slow` with an explicit inner index loop and re-time. The class does not change — explain precisely what *does* change and by roughly what factor.
4. Build a list of 50,000 items twice: once with `append`, once with `insert(0, ...)`. Time both. Explain the result using this lecture's table.
5. Reading: CLRS §2.2. Half a page on why counting “basic operations” is legitimate even though different operations take different amounts of real time.

Next session · Tuesday 6 October: Best, worst and average case — and the distinction between the *case* you analyse and the *bound* you use, which is the single most common source of confusion in this subject. Then a full case study: the Two-Sum problem taken from $\Theta(n^2)$ to $\Theta(n)$.